

MCP, Deployment & Production AI Systems

From prototype to scalable, secure, observable AI products

What This Module Is About

Most AI prototypes start with a handful of Python scripts duct-taped together until something works. That approach ships demos — but it is a liability in production. This module teaches you to build with production architecture from the start: MCP protocol, CI/CD pipelines, security best practices, and reliability engineering.

What Is MCP (Model Context Protocol)?

MCP is a standardised protocol for communication between AI system components. Instead of ad-hoc integrations, MCP defines clear interfaces: each component has a defined responsibility, a defined input/output schema, and a defined error contract. This makes systems modular, testable, and replaceable.

The Deployment Pipeline

Step	Action	Tooling
1. Commit	Code versioned and pushed	Git, GitHub
2. CI — Test	Automated unit + integration tests run	GitHub Actions, pytest
3. Build	Docker image built and tagged	Docker, GitHub Container Registry
4. Staging Deploy	Deploy to staging environment	Railway, Render, or ECS
5. Canary	Gradual rollout to % of real traffic	Load balancer, feature flags
6. Prod Deploy	Full production rollout	Rolling update, zero downtime
7. Monitor	Observe metrics, logs, alerts	Datadog, Grafana, CloudWatch

Security Essentials

- **Least Privilege** — each service gets only the permissions it needs. Your TTS service should not have write access to your database.
- **Secret Management** — use environment variables or a secrets manager (AWS Secrets Manager, HashiCorp Vault). Never commit credentials to Git.

- **API Key Isolation** — create separate API keys per service/environment. Rotate keys immediately if one is leaked.
- **Audit Logging** — log every external API call with timestamp, service, and outcome. This is your forensic trail when something goes wrong.
- **Input Validation** — validate and sanitise all user inputs before passing to your LLM. Prompt injection is a real attack vector.

Tip: Use Railway or Render for your first production deployment — they handle most of the infrastructure complexity so you can focus on your AI system logic.

Resources for Deeper Learning

1 Model Context Protocol — Official Docs

<https://modelcontextprotocol.io/introduction>

The official MCP specification and introduction. Read this to understand the protocol's design goals, how components communicate, and how to implement MCP-compliant services.

2 Docker — Getting Started

<https://docs.docker.com/get-started/>

Docker is the foundation of modern deployment. Learn to containerise your FastAPI + AI application so it runs identically in development, staging, and production.

3 GitHub Actions Documentation

<https://docs.github.com/en/actions>

GitHub Actions is your CI/CD system. Learn to write workflows that automatically run tests, build Docker images, and deploy to your cloud platform on every push.

4 Railway — Deployment Platform

<https://docs.railway.app/>

Railway is one of the simplest ways to deploy containerised Python apps. Connect your GitHub repo, add environment variables, and you have a live URL in minutes.

5 OWASP API Security Top 10

<https://owasp.org/www-project-api-security/>

The OWASP API Security Top 10 lists the most common API vulnerabilities. Read this to understand what attackers look for in AI APIs and how to defend against them.

6 Prometheus + Grafana — Monitoring

<https://prometheus.io/docs/introduction/overview/>

Prometheus scrapes metrics from your application; Grafana visualises them. This stack gives you dashboards for request rate, latency, error rate, and custom AI metrics.

Module 5 Assignment — Containerise and Deploy a Component

- Write a Dockerfile for your FastAPI + Claude API service from Module 2. Build the image locally and confirm it runs with 'docker run'.
- Create a GitHub Actions workflow (.github/workflows/deploy.yml) that runs your pytest tests automatically on every push to main.
- Deploy your containerised service to Railway or Render. Confirm it has a public HTTPS URL and responds to requests from the internet.
- Add a /health endpoint to your API that returns {status: ok, version: '1.0'}. Verify it is accessible on your public URL.
- Audit your code for secrets: search for any hardcoded API keys or passwords. Move everything to environment variables. Document your env vars in README.md (names only, not values).

Note: After this module, you have a deployed service. Your capstone will build on top of it.